OCOM5300M Artificial Intelligence Project Cross Attention LSTM Decoder

In [1]:
```
# Custom Classes
from captions import Captions
from vocabulary import Vocabulary
from dataset_concept import ROCOv2_Subset, ConceptLookup, ConceptVocabulary
from pipeline_concept import AdditiveCrossAttention, Decoder, DecoderModel

# Deep learning Functionality
import torch  # PyTorch library
import torch.nn as nn  # PyTorch neural network module
from torch.utils.data import Dataset, DataLoader  # creating custom data containers, batching, shuffling and data
loading capabilities
import torch.nn.functional as F  # PyTorch functional interface e.g., activation functions, loss calculations
import torch.optim as optim  # optimization algorithms for training neural networks

# Data Handling And Analysis
import pandas as pd  # data manipulation and analysis using DataFrame structures
from collections import Counter  # counting occurrences, e.g., word frequencies

# Visualization
import matplotlib.pyplot as plt  # plotting and visualizing results
import matplotlib.gridspec as gridspec #creating grid-based layouts of subplots
from tqdm import tqdm # progress bars

# Utilities
import os  # file and directory operations
import time  # timing operations
import re #regular expressions
import math # common mathematical functions
from pathlib import Path # file and directory operations
```

In [2]:
```
# suppress Intel specific Math Kernel Library (MKL) warnings
os.environ['KMP_DUPLICATE_LIB_OK'] = 'True'
```

In [3]:
```
# device
device = torch.device('cpu')
```

In [4]:
```
# display the complete content of DataFrame columns without truncation
pd.set_option('display.max_colwidth', None);
```

In [5]:
```
# set random seed to 42
seed = 42
```

In [6]:
```
# seed set to ensure reproducibility of results
torch.manual_seed(seed)
```

Out[6]:
```
<torch._C.Generator at 0x2b82e6458b0>
```

Data Preparation

In [7]:
```
# training captions filename
training_captions_filename = "ROCOv2/train_captions.csv"
```

In [8]:
```
# instantiate training captions object
training_captions = Captions()
```

In [9]:
```
# load training captions
training_caption_data = training_captions.load_captions(training_captions_filename)
```

In [10]:
```
# clean training captions
training_captions.clean_captions();
```

In [11]:
```
# confirm number of cleaned training captions
len(training_caption_data)
```

Out[11]:
```
59958
```

In [12]:
```
# maximum caption length
maximum_caption_length = (training_captions.get_caption_length_percentile(99))
```

In [13]:
```
# validation captions filename
valid_captions_filename = "ROCOv2/valid_captions.csv"
```

In [14]:
```
# instantiate validation captions object
valid_captions = Captions()
```

In [15]:
```
# load validation captions
valid_caption_data = valid_captions.load_captions(valid_captions_filename)
```

In [16]:
```
# clean validation captions
valid_captions.clean_captions();
```

In [17]:
```
# confirm number of cleaned validation captions
len(valid_caption_data)
```

Out[17]:
```
9904
```

In [18]:
```
# instantiate Vocabulary object
vocabulary = Vocabulary()
```

In [19]:
```
# reload vocabulary
vocabulary = Vocabulary.load("ROCOv2/vocabulary.pt")
```

In [20]:
```
# confirm number of tokens in vocabulary, including special tokens
len(vocabulary)
```

Out[20]:
```
17602
```

In [21]:
```
# set batch size to 32 for data loading
batch_size = 32
```

In [22]:
```
# pipeline CUI configuration
pipeline_training_filename = "ROCOv2/pipeline_training.csv"
pipeline_validation_filename = "ROCOv2/pipeline_validation.csv"
concept_vocabulary_filename = "lstm_pipeline_cui_vocabulary.json"

# create CUI vocabulary using training data only
concept_vocabulary = ConceptVocabulary.from_training_csv(pipeline_training_filename)
concept_vocabulary.save(concept_vocabulary_filename)
train_concepts = ConceptLookup(pipeline_training_filename, concept_vocabulary, max_concepts=20)
valid_concepts = ConceptLookup(pipeline_validation_filename, concept_vocabulary, max_concepts=20)
```

```
print("CUI vocabulary size:", len(concept_vocabulary))
print("Train images with CUIs:",len(train_concepts.by_image))
print("Valid images with CUIs:",len(valid_concepts.by_image))
```

```
CUI vocabulary size: 1810
Train images with CUIs: 59846
Valid images with CUIs: 4943
```

In [23]:
```
# feature files folder for training images
training_feature_folder = (r"ROCOv2\train_images\train\pt")
```

In [24]:
```
# create training dataset
train_dataset = ROCOv2_Subset(
    training_caption_data,
    training_feature_folder,
    vocabulary,
    maximum_caption_length,
    concept_lookup=train_concepts,
)
```

In [25]:
```
# training set dataloader
train_loader = DataLoader(
        train_dataset,
        batch_size=batch_size,
        shuffle=True # set shuffle = true for the training set data loader
    )
```

In [26]:
```
# feature files folder for training images
valid_feature_folder = (r"ROCOv2\valid_images\valid\pt")
```

In [27]:
```
# create validation dataset
valid_dataset = ROCOv2_Subset(
    valid_caption_data,
    valid_feature_folder,
    vocabulary,
    maximum_caption_length,
    concept_lookup=valid_concepts,
)
```

In [28]:
```
# validation set dataloader
valid_loader = DataLoader(
        valid_dataset,
        batch_size=batch_size,
        shuffle=False # set shuffle = false
)
```

Decoder

In [29]:
```
# set hyperparameter, i.e., the dimensionality of the word embedding vectors in the model
embed_size = 256
```

In [30]:
```
# set hyperparameter, i.e., dimensionality of the hidden state in LSTM layers
hidden_size = 512
```

In [31]:
```
# set hyperparameter, i.e., the number of stacked recurrent layers in the neural network architecture
number_of_layers = 1
```

In [32]:
```
# set encoder feature dimension
encoder_dimension = 2048
```

In [33]:
```
# set attention dimension, i.e., the size of the intermediate feature space used to calculate attention scores
# for relevancy of each image region when the decoder is generating the next word
attention_dimension = 512
```

In [34]:
```
# specify the saved best-performing model file, i.e., based on the epoch with the lowest validation loss
selected_model = 'cross_attention_lstm_pipeline_best.pth'
```

In [35]:
```
# initial step size when updating the model weights
learning_rate = 0.0015
```

In [36]:
```
# size of penalty to be applied large model weights to help reduce overfitting.
weight_decay = 1e-4
```

In [37]:
```
# maximum gradient size allowed before clipping to help prevent unstable weight updates
maximum_gradient_size = 1.0
```

In [38]:
```
#number of epochs
number_of_epochs = 20
```

In [39]:
```
# early stopping patience
early_stopping_patience = 6
```

In [40]:
```
# dropout, i.e., 30% of the selected neuron activations are randomly set to zero on each forward pass.
# This forces the model not to rely too heavily on particular neurons
# and can improve generalisation to unseen data to prevent overfitting
dropout = 0.3
```

In [41]:
```
# select visual gate type: scalar, vector or none
visual_gate_type = "none"
```

In [42]:
```
# numerical index assigned to the <pad> token in the vocabulary
pad_idx = vocabulary("<pad>")
```

In [43]:
```
# create decoder model
decoder = DecoderModel(
    vocab_size=len(vocabulary),
    pad_idx=pad_idx,
    encoder_dim=encoder_dimension,
    embed_size=embed_size,
    hidden_size=hidden_size,
    attention_dim=attention_dimension,
    num_layers=number_of_layers,
    max_seq_length=maximum_caption_length,
    dropout=dropout,
    concept_vocab_size=len(concept_vocabulary),
    visual_gate_type=visual_gate_type
)
```

In [44]:
```
# train model
model_statistics = decoder.train_model(
    train_loader=train_loader,
    val_loader=valid_loader,
    num_epochs=number_of_epochs,
    learning_rate=learning_rate,
    weight_decay=weight_decay,
```

```
    patience=early_stopping_patience,
    clip_grad_norm=maximum_gradient_size,
    checkpoint_path=selected_model,
    device=device
)
```

```
Cross-attention LSTM training 1/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 1/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [1/20] - 6022.58s
Training loss: 5.4005 | Training perplexity: 221.5149
Validation loss: 4.6777 | Validation perplexity: 107.5183
Learning rate: 0.001500
Overfitting gap: -0.7228
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 2/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 2/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [2/20] - 7519.73s
Training loss: 4.5334 | Training perplexity: 93.0758
Validation loss: 4.3745 | Validation perplexity: 79.4000
Learning rate: 0.001500
Overfitting gap: -0.1589
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 3/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 3/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [3/20] - 14437.86s
Training loss: 4.3498 | Training perplexity: 77.4593
Validation loss: 4.2704 | Validation perplexity: 71.5484
Learning rate: 0.001500
Overfitting gap: -0.0794
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 4/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 4/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [4/20] - 8203.69s
Training loss: 4.2638 | Training perplexity: 71.0830
Validation loss: 4.2184 | Validation perplexity: 67.9240
Learning rate: 0.001500
Overfitting gap: -0.0455
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 5/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 5/20:   0%|          | 0/310 [00:00<?, ?it/s]
```

```
Epoch [5/20] - 6127.51s
Training loss: 4.2114 | Training perplexity: 67.4529
Validation loss: 4.1955 | Validation perplexity: 66.3862
Learning rate: 0.001500
Overfitting gap: -0.0159
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 6/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 6/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [6/20] - 6086.40s
Training loss: 4.1771 | Training perplexity: 65.1735
Validation loss: 4.1611 | Validation perplexity: 64.1447
Learning rate: 0.001500
Overfitting gap: -0.0159
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 7/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 7/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [7/20] - 6104.22s
Training loss: 4.1533 | Training perplexity: 63.6435
Validation loss: 4.1443 | Validation perplexity: 63.0758
Learning rate: 0.001500
Overfitting gap: -0.0090
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 8/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 8/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [8/20] - 6077.70s
Training loss: 4.1320 | Training perplexity: 62.3027
Validation loss: 4.1228 | Validation perplexity: 61.7325
Learning rate: 0.001500
Overfitting gap: -0.0092
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 9/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 9/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [9/20] - 6071.95s
Training loss: 4.1126 | Training perplexity: 61.1074
Validation loss: 4.1172 | Validation perplexity: 61.3869
Learning rate: 0.001500
Overfitting gap: 0.0046
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 10/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 10/20:   0%|          | 0/310 [00:00<?, ?it/s]
```

```
Epoch [10/20] - 6075.28s
Training loss: 4.0999 | Training perplexity: 60.3360
Validation loss: 4.1077 | Validation perplexity: 60.8056
Learning rate: 0.001500
Overfitting gap: 0.0078
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 11/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 11/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [11/20] - 6098.83s
Training loss: 4.0857 | Training perplexity: 59.4846
Validation loss: 4.0985 | Validation perplexity: 60.2517
Learning rate: 0.001500
Overfitting gap: 0.0128
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 12/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 12/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [12/20] - 6089.51s
Training loss: 4.0740 | Training perplexity: 58.7888
Validation loss: 4.0843 | Validation perplexity: 59.3992
Learning rate: 0.001500
Overfitting gap: 0.0103
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 13/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 13/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [13/20] - 6110.32s
Training loss: 4.0617 | Training perplexity: 58.0749
Validation loss: 4.0750 | Validation perplexity: 58.8479
Learning rate: 0.001500
Overfitting gap: 0.0132
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 14/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 14/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [14/20] - 6142.07s
Training loss: 4.0529 | Training perplexity: 57.5628
Validation loss: 4.0691 | Validation perplexity: 58.5034
Learning rate: 0.001500
Overfitting gap: 0.0162
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 15/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 15/20:   0%|          | 0/310 [00:00<?, ?it/s]
```

```
Epoch [15/20] - 6140.68s
Training loss: 4.0439 | Training perplexity: 57.0497
Validation loss: 4.0667 | Validation perplexity: 58.3663
Learning rate: 0.001500
Overfitting gap: 0.0228
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 16/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 16/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [16/20] - 6141.59s
Training loss: 4.0325 | Training perplexity: 56.3997
Validation loss: 4.0503 | Validation perplexity: 57.4160
Learning rate: 0.001500
Overfitting gap: 0.0179
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 17/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 17/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [17/20] - 6146.22s
Training loss: 4.0249 | Training perplexity: 55.9731
Validation loss: 4.0517 | Validation perplexity: 57.4949
Learning rate: 0.001500
Overfitting gap: 0.0268
No validation improvement for 1 epoch(s).
Cross-attention LSTM training 18/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 18/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [18/20] - 6155.09s
Training loss: 4.0173 | Training perplexity: 55.5484
Validation loss: 4.0432 | Validation perplexity: 57.0069
Learning rate: 0.001500
Overfitting gap: 0.0259
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Cross-attention LSTM training 19/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 19/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [19/20] - 6234.74s
Training loss: 4.0115 | Training perplexity: 55.2293
Validation loss: 4.0545 | Validation perplexity: 57.6591
Learning rate: 0.001500
Overfitting gap: 0.0431
No validation improvement for 1 epoch(s).
Cross-attention LSTM training 20/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 20/20:   0%|          | 0/310 [00:00<?, ?it/s]
```

```
Epoch [20/20] - 6620.93s
Training loss: 4.0041 | Training perplexity: 54.8243
Validation loss: 4.0318 | Validation perplexity: 56.3598
Learning rate: 0.001500
Overfitting gap: 0.0276
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_pipeline_best.pth'.
Best validation loss: 4.0318 at epoch 20.
```

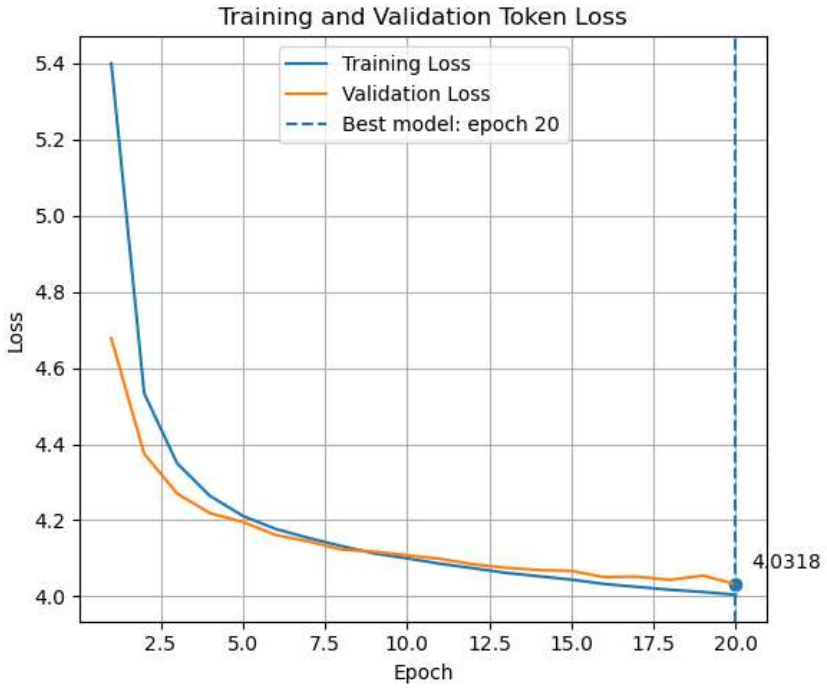
Diagnostics

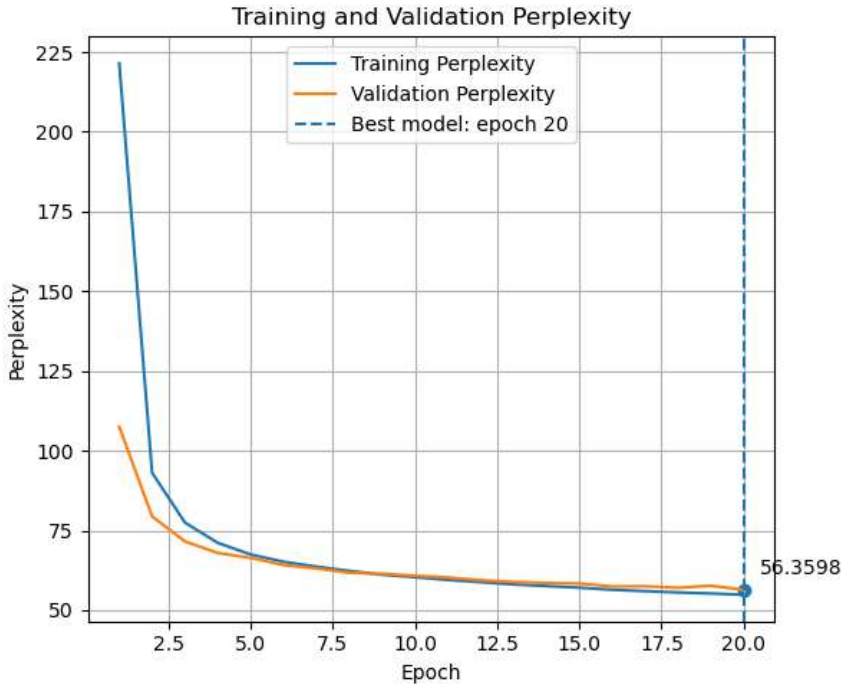
In [45]:
```
# display model's epoch statistics
model_statistics
```

Out[45]:

	epoch	training loss	validation loss	training perplexity	validation perplexity	learning rate	overfitting
0	1	5.400490	4.677661	221.514925	107.518304	0.0015	-0.722829
1	2	4.533414	4.374498	93.075792	79.400006	0.0015	-0.158916
2	3	4.349753	4.270375	77.459302	71.548444	0.0015	-0.079378
3	4	4.263848	4.218390	71.082975	67.924023	0.0015	-0.045458
4	5	4.211430	4.195489	67.452904	66.386155	0.0015	-0.015941
5	6	4.177054	4.161142	65.173548	64.144739	0.0015	-0.015912
6	7	4.153297	4.144337	63.643515	63.075782	0.0015	-0.008961
7	8	4.132005	4.122811	62.302700	61.732541	0.0015	-0.009194
8	9	4.112633	4.117197	61.107422	61.386939	0.0015	0.004564
9	10	4.099929	4.107682	60.336030	60.805620	0.0015	0.007753
10	11	4.085718	4.098531	59.484644	60.251734	0.0015	0.012813
11	12	4.073951	4.084280	58.788792	59.399164	0.0015	0.010329
12	13	4.061733	4.074956	58.074852	58.847903	0.0015	0.013223
13	14	4.052876	4.069085	57.562764	58.503386	0.0015	0.016209
14	15	4.043923	4.066739	57.049719	58.366296	0.0015	0.022815
15	16	4.032464	4.050323	56.399702	57.416011	0.0015	0.017859
16	17	4.024871	4.051696	55.973063	57.494888	0.0015	0.026825
17	18	4.017255	4.043172	55.548426	57.006875	0.0015	0.025917
18	19	4.011494	4.054548	55.229320	57.659108	0.0015	0.043054
19	20	4.004134	4.031756	54.824315	56.359789	0.0015	0.027622

In [46]:
```
# display performance graphs
decoder.plot_model_performance()
```


Training and Validation Token Loss
Training Loss
Validation Loss
Best model: epoch 20
Loss
Epoch
4.0318


Training and Validation Perplexity
Training Perplexity
Validation Perplexity
Best model: epoch 20
Perplexity
Epoch
56.3598

Out[46]:

```
DecoderModel(
  (concept_embeddings): Embedding(1810, 256, padding_idx=0)
  (concept_projection): Linear(in_features=256, out_features=2048, bias=False)
  (embedding): Embedding(17602, 256, padding_idx=0)
  (cross_attention): AdditiveCrossAttention(
    (visual_projection): Linear(in_features=2048, out_features=512, bias=True)
    (hidden_projection): Linear(in_features=512, out_features=512, bias=True)
    (attention_score): Linear(in_features=512, out_features=1, bias=False)
  )
  (lstm_cells): ModuleList(
    (0): LSTMCell(2304, 512)
  )
  (initial_hidden): ModuleList(
    (0): Linear(in_features=2048, out_features=512, bias=True)
  )
  (initial_cell): ModuleList(
    (0): Linear(in_features=2048, out_features=512, bias=True)
  )
  (visual_gate): Linear(in_features=512, out_features=1, bias=True)
  (dropout): Dropout(p=0.3, inplace=False)
  (hidden_to_embed): Linear(in_features=512, out_features=256, bias=True)
  (output_layer): Linear(in_features=256, out_features=17602, bias=False)
)
```

In [47]:

```python
def plot_gate_diagnostics(decoder, data_loader, device):
    """
    Calculate and plot the mean visual gate value at each caption position.

    Args:
        decoder: trained decoder model
        data_loader: validation DataLoader
        device: device used for model calculations

    Returns:
        pd.DataFrame: gate diagnostics by caption timestep
    """

    try:

        # calculate mean visual gate values by caption position
        gate_diagnostics = decoder.calculate_gate_mean_by_timestep(
            data_loader,
```

```
            device
        )

        # create figure
        plt.figure(figsize=(10, 5))

        # plot mean visual gate by caption position
        plt.plot(
            gate_diagnostics["Timestep"],
            gate_diagnostics["Mean Gate"],
            marker="o"
        )

        # configure plot
        plt.xlabel("Caption Position")
        plt.ylabel("Mean Visual Gate")
        plt.title("Mean Visual Gate by Caption Position")
        plt.ylim(0, 1)
        plt.grid(True)

        # display plot
        plt.show()

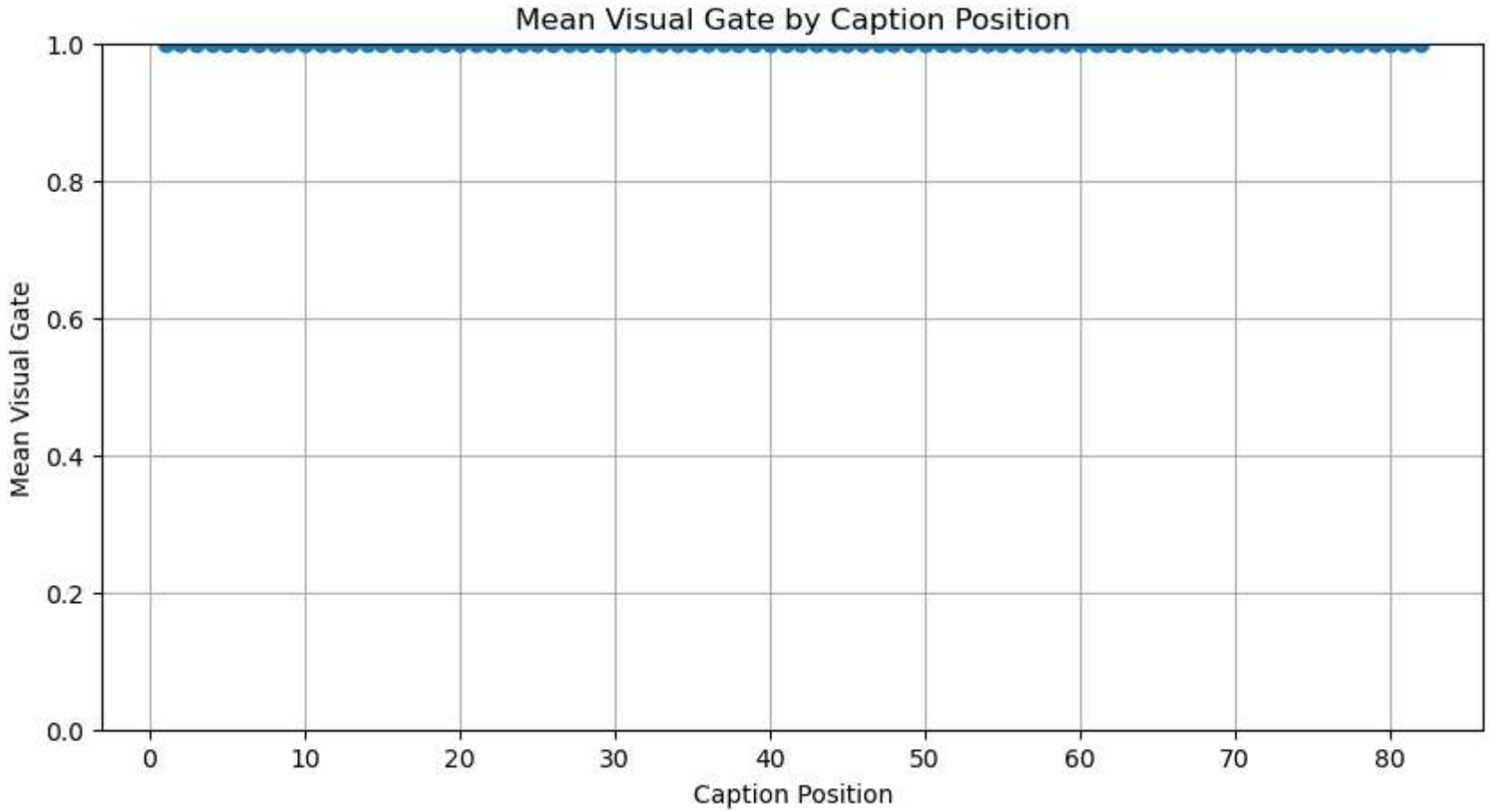
        return gate_diagnostics

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [48]:

```
# the gate is a learned control value that regulates how much visual information from the image is allowed into the decoder at each caption timestep.
# calculate mean visual gate value by caption timestep
gate_diagnostics = plot_gate_diagnostics(decoder, valid_loader, device)
```

```
Gate diagnostics:   0%|          | 0/310 [00:00<?, ?it/s]
```



In [49]:
```
# display gate diagnostics
display(gate_diagnostics)
```

	Timestep	Mean Gate	Valid Captions
0	1	1.0	9904.0
1	2	1.0	9903.0
2	3	1.0	9898.0
3	4	1.0	9826.0
4	5	1.0	9715.0
...	...	...	...
77	78	1.0	186.0
78	79	1.0	177.0
79	80	1.0	170.0
80	81	1.0	162.0
81	82	1.0	152.0

82 rows × 3 columns

In [50]:
```python
def plot_visual_context_diagnostics(decoder, data_loader, device):
    """
    Calculate and plot the mean visual context magnitude
    at each caption position.

    Args:
        decoder: trained decoder model
        data_loader: validation DataLoader
        device: device used for model calculations

    Returns:
        pd.DataFrame: visual context diagnostics by caption timestep
    """

    try:

        # calculate mean visual context magnitude by caption position
```

```
        visual_context_diagnostics = (
            decoder.calculate_visual_context_magnitude_by_timestep(
                data_loader,
                device
            )
        )

        # create figure
        plt.figure(figsize=(10, 5))

        # plot mean visual context magnitude by caption position
        plt.plot(
            visual_context_diagnostics["Timestep"],
            visual_context_diagnostics["Mean Visual Context Magnitude"],
            marker="o"
        )

        # configure plot
        plt.xlabel("Caption Position")
        plt.ylabel("Mean Visual Context Magnitude")
        plt.title("Mean Visual Context Magnitude by Caption Position")
        plt.grid(True)

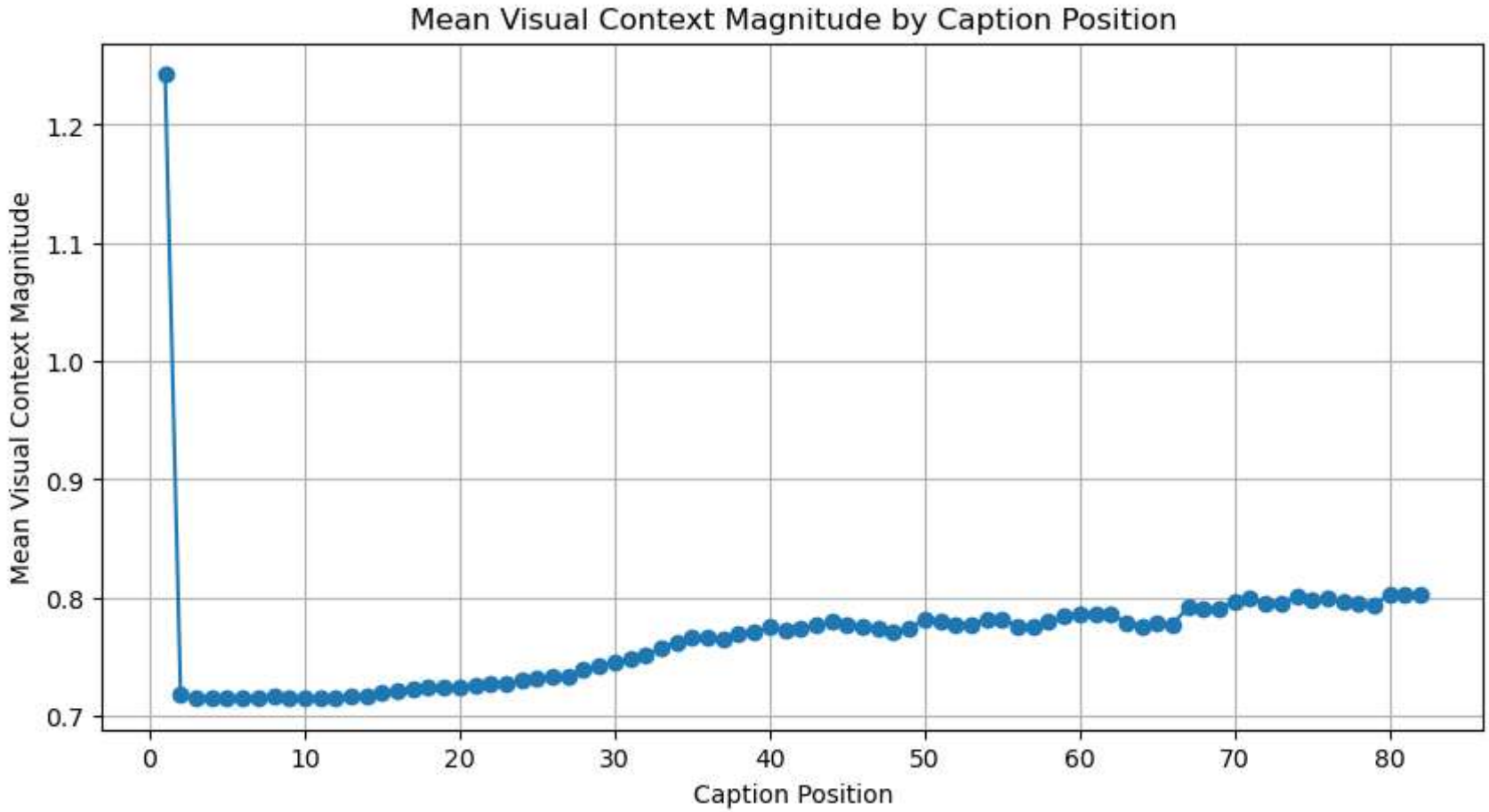
        # display plot
        plt.show()

        return visual_context_diagnostics

    except Exception as e:
            raise RuntimeError(f"Exception : {str(e)}")
```

In [51]:
```
# visual context magnitude
visual_context_diagnostics = plot_visual_context_diagnostics(decoder, valid_loader, device)
```

```
Visual Context Diagnostics:   0%|          | 0/310 [00:00<?, ?it/s]
```



In [52]:
```
# display visual context magnitude
display(visual_context_diagnostics)
```

	Timestep	Mean Visual Context Magnitude	Valid Captions
0	1	1.242498	9904.0
1	2	0.718094	9903.0
2	3	0.715298	9898.0
3	4	0.715233	9826.0
4	5	0.715242	9715.0
...	...	...	...
77	78	0.795069	186.0
78	79	0.793717	177.0
79	80	0.803172	170.0
80	81	0.802396	162.0
81	82	0.803064	152.0

82 rows × 3 columns

In [53]:
```
def plot_attention_entropy_diagnostics(
    decoder,
    data_loader,
    device
):
    """
    Calculate and plot normalized attention entropy
    at each caption position.

    Args:
        decoder: trained decoder model
        data_loader: validation DataLoader
        device: device used for model calculations

    Returns:
        pd.DataFrame: attention entropy diagnostics
        by caption timestep
```

```
    """
    try:

        # calculate attention entropy by caption position
        attention_entropy_diagnostics = (
            decoder.calculate_attention_entropy_by_timestep(
                data_loader,
                device
            )
        )

        # create figure
        plt.figure(figsize=(10, 5))

        # plot normalized attention entropy
        plt.plot(
            attention_entropy_diagnostics["Timestep"],
            attention_entropy_diagnostics[
                "Normalized Attention Entropy"
            ],
            marker="o"
        )

        # configure plot
        plt.xlabel("Caption Position")
        plt.ylabel("Normalized Attention Entropy")
        plt.title("Attention Entropy by Caption Position")

        # normalized entropy ranges from 0 to 1
        plt.ylim(0, 1)
        plt.grid(True)

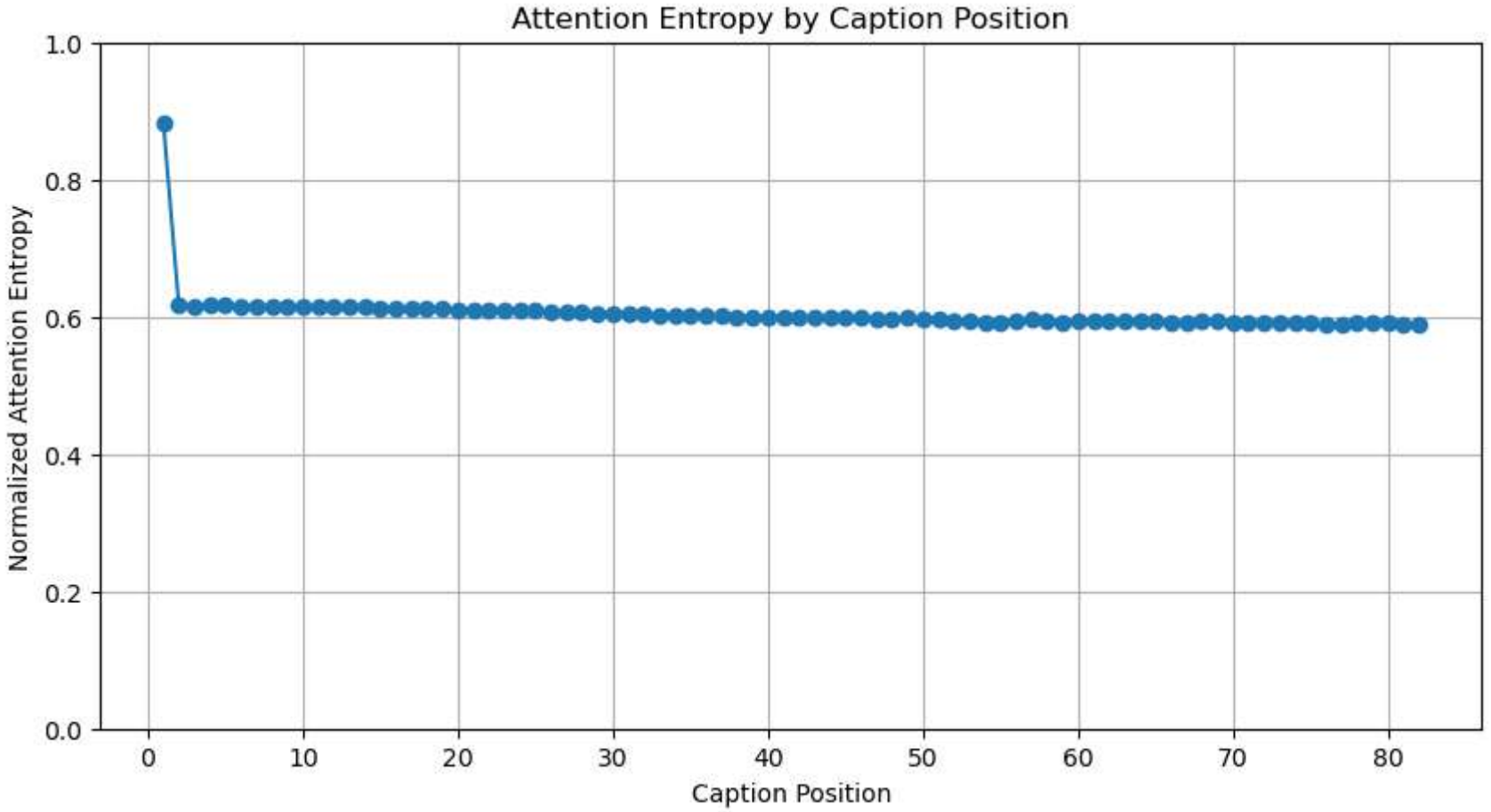
        # display plot
        plt.show()

        return attention_entropy_diagnostics

    except Exception as e:
            raise RuntimeError(f"Exception : {str(e)}")
```

In [54]:
```
# attention entropy shows how concentrated or spread out the model's visual attention is across the image regions
at each caption position
# attention entropy by timestep
attention_entropy_diagnostics = plot_attention_entropy_diagnostics(decoder, valid_loader, device)
```

```
Attention entropy diagnostics:   0%|          | 0/310 [00:00<?, ?it/s]
```



In [55]:
```
# display attention entropy diagnostics
display(attention_entropy_diagnostics)
```

	Timestep	Mean Attention Entropy	Normalized Attention Entropy	Valid Captions
0	1	3.437642	0.883299	9904.0
1	2	2.398611	0.616321	9903.0
2	3	2.397466	0.616027	9898.0
3	4	2.405499	0.618091	9826.0
4	5	2.398481	0.616288	9715.0
...	...	...	...	...
77	78	2.299356	0.590818	186.0
78	79	2.301054	0.591254	177.0
79	80	2.298950	0.590713	170.0
80	81	2.291662	0.588841	162.0
81	82	2.292997	0.589184	152.0

82 rows × 4 columns

In [56]:
```
# calculate the out-of-vocabulary statistics for the validation captions against the final vocabulary
valid_oov = vocabulary.calculate_oov_statistics(valid_caption_data, dataset_name="Validation")
```

In [57]:
```
def plot_oov_statistics(oov_statistics):
    """
    Display token-level and caption-level out-of-vocabulary (OOV) statistics
    as horizontal percentage-based stacked bar charts with summary tables

    Args:
        oov_statistics (dict): OOV statistics returned by calculate_oov_statistics()
    """
    try:

        # retrieve dataset name
        dataset_name = oov_statistics["dataset"]
```

```
        # retrieve total number of tokens
        total_tokens = oov_statistics["total_tokens"]

        # retrieve total number of out-of-vocabulary tokens
        oov_tokens = oov_statistics["oov_tokens"]

        # retrieve precentage of tokens out-of-vocabulary
        token_oov_percentage = oov_statistics["oov_rate"]

        # calculate percentage of tokens that are present in the vocabulary
        in_vocabulary_percentage = 100 - token_oov_percentage

        # retrieve total number of captions
        total_captions = oov_statistics["total_captions"]

        # retrieve number of captions containing at least one out-of-vocabulary token
        captions_with_oov = oov_statistics["captions_with_oov"]

        # retrieve percentage of captions containing at least one out-of-vocabulary token
        caption_oov_percentage = oov_statistics["oov_caption_percentage"]

        # calculate percentage of captions containing no OOV words
        captions_without_oov_percentage = 100 - caption_oov_percentage

        # create vertically stacked charts
        fig, axes = plt.subplots(2, 1, figsize=(10, 5))

        # plot the percentage of tokens that are present in the vocabulary
        axes[0].barh(0, in_vocabulary_percentage, height=0.25, color="black", edgecolor="black", linewidth=0.8)

        # plot the percentage of out-of-vocabulary tokens
        axes[0].barh(0, token_oov_percentage, left=in_vocabulary_percentage, height=0.25, color="lightgrey",
edgecolor="grey", linewidth=0.8, hatch="//")

        # set token-level chart title
        axes[0].set_title(f'{dataset_name} Token-Level OOV', loc="left", pad=10)

        # display token category labels centred above each bar segment
        axes[0].text(in_vocabulary_percentage / 2, 0.22, "In Vocabulary", ha="center", va="bottom")
        axes[0].text(in_vocabulary_percentage + (token_oov_percentage / 2), 0.22, "OOV", ha="center", va="bottom")

        # create token-level summary table
```

```
        token_table = axes[0].table(cellText=[[f"{total_tokens:,}", f"{oov_tokens:,}", f"
{token_oov_percentage:.2f}%"]],
                                    colLabels=["Total Tokens", "OOV Tokens", "OOV Rate (%)"], cellLoc="center",
loc="bottom",
                                    bbox=[0, -0.42, 1, 0.30])

        # set token-level table borders
        for cell in token_table.get_celld().values():
            cell.set_linewidth(0.8)

        # plot the percentage of captions containing no OOV words
        axes[1].barh(0, captions_without_oov_percentage, height=0.25, color="black", edgecolor="black",
linewidth=0.8)

        # plot the percentage of captions containing at least one OOV word
        axes[1].barh(0, caption_oov_percentage, left=captions_without_oov_percentage, height=0.25,
                     color="lightgrey", edgecolor="grey", linewidth=0.8, hatch="//")

        # set caption-level chart title
        axes[1].set_title(f'{dataset_name} Caption-Level OOV', loc="left", pad=10)

        # display caption category labels centred above each bar segment
        axes[1].text(captions_without_oov_percentage / 2, 0.22, "Without OOV", ha="center", va="bottom")
        axes[1].text(captions_without_oov_percentage + (caption_oov_percentage / 2), 0.22, "With OOV", ha="center",
va="bottom")

        # create caption-level summary table
        caption_table = axes[1].table(cellText=[[f"{total_captions:,}", f"{captions_with_oov:,}", f"
{caption_oov_percentage:.2f}%"]],
                                      colLabels=["Total Captions", "Captions With OOV Word", "OOV Caption (%)"],
cellLoc="center", loc="bottom",
                                      bbox=[0, -0.42, 1, 0.30])

        # set caption-level table borders
        for cell in caption_table.get_celld().values():
            cell.set_linewidth(0.8)

        # remove axes, tick marks and borders
        for axis in axes:

            # set percentage scale
            axis.set_xlim(0, 100)
```

```
            # set vertical spacing
            axis.set_ylim(-0.45, 0.55)

            # remove axis tick marks
            axis.set_xticks([])
            axis.set_yticks([])

            # remove chart borders
            for spine in axis.spines.values():
                spine.set_visible(False)

        # reduce spacing between charts
        plt.subplots_adjust(hspace=0.75)

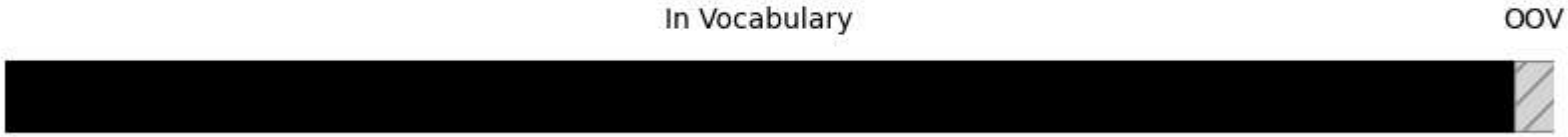
        # display charts
        plt.show()

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [58]:

```
# display OOV statistics graphically
plot_oov_statistics(valid_oov)
```

Validation Token-Level OOV



Total Tokens	OOV Tokens	OOV Rate (%)
227,293	5,825	2.56%

Validation Caption-Level OOV



Total Captions	Captions With OOV Word	OOV Caption (%)
9,904	3,082	31.12%

In [59]:
```
def plot_caption_length_comparison(training_captions, valid_captions):
    """
    Calculate, summarise and plot training and validation caption-length statistics

    Args:
        training_captions: cleaned training Captions object
        valid_captions: cleaned validation Captions object

    Returns:
        tuple:
            training_caption_lengths: training caption lengths
            validation_caption_lengths: validation caption lengths
```

```
        caption_length_summary: summary statistics for training and validation caption lengths
    """

    try:

        # calculate training caption lengths
        training_caption_lengths = (
            training_captions.calculate_caption_lengths()
        )

        # calculate validation caption lengths
        validation_caption_lengths = (
            valid_captions.calculate_caption_lengths()
        )

        # summarise training and validation caption lengths
        caption_length_summary = pd.DataFrame([
            {
                "Dataset": "Training",
                "Captions": len(training_caption_lengths),
                "Mean Length": training_caption_lengths.mean(),
                "Median Length": training_caption_lengths.median(),
                "Minimum Length": training_caption_lengths.min(),
                "Maximum Length": training_caption_lengths.max(),
                "Std Dev": training_caption_lengths.std(),
            },
            {
                "Dataset": "Validation",
                "Captions": len(validation_caption_lengths),
                "Mean Length": validation_caption_lengths.mean(),
                "Median Length": validation_caption_lengths.median(),
                "Minimum Length": validation_caption_lengths.min(),
                "Maximum Length": validation_caption_lengths.max(),
                "Std Dev": validation_caption_lengths.std(),
            }
        ])

        # select caption-length statistics for graphical comparison
        length_statistics = (
            caption_length_summary
            .set_index("Dataset")[
                [
```

```
                    "Mean Length",
                    "Median Length",
                    "Minimum Length",
                    "Maximum Length",
                    "Std Dev"
                ]
            ]
        )

        # create grouped bar chart
        length_statistics.T.plot(
            kind="bar",
            figsize=(10, 5)
        )

        # configure plot
        plt.xlabel("Caption Length Statistic")
        plt.ylabel("Caption Length (words)")
        plt.title("Training and Validation Caption Length Comparison")
        plt.xticks(rotation=45)
        plt.legend(title="Dataset")
        plt.grid(axis="y")

        # prevent labels from overlapping
        plt.tight_layout()

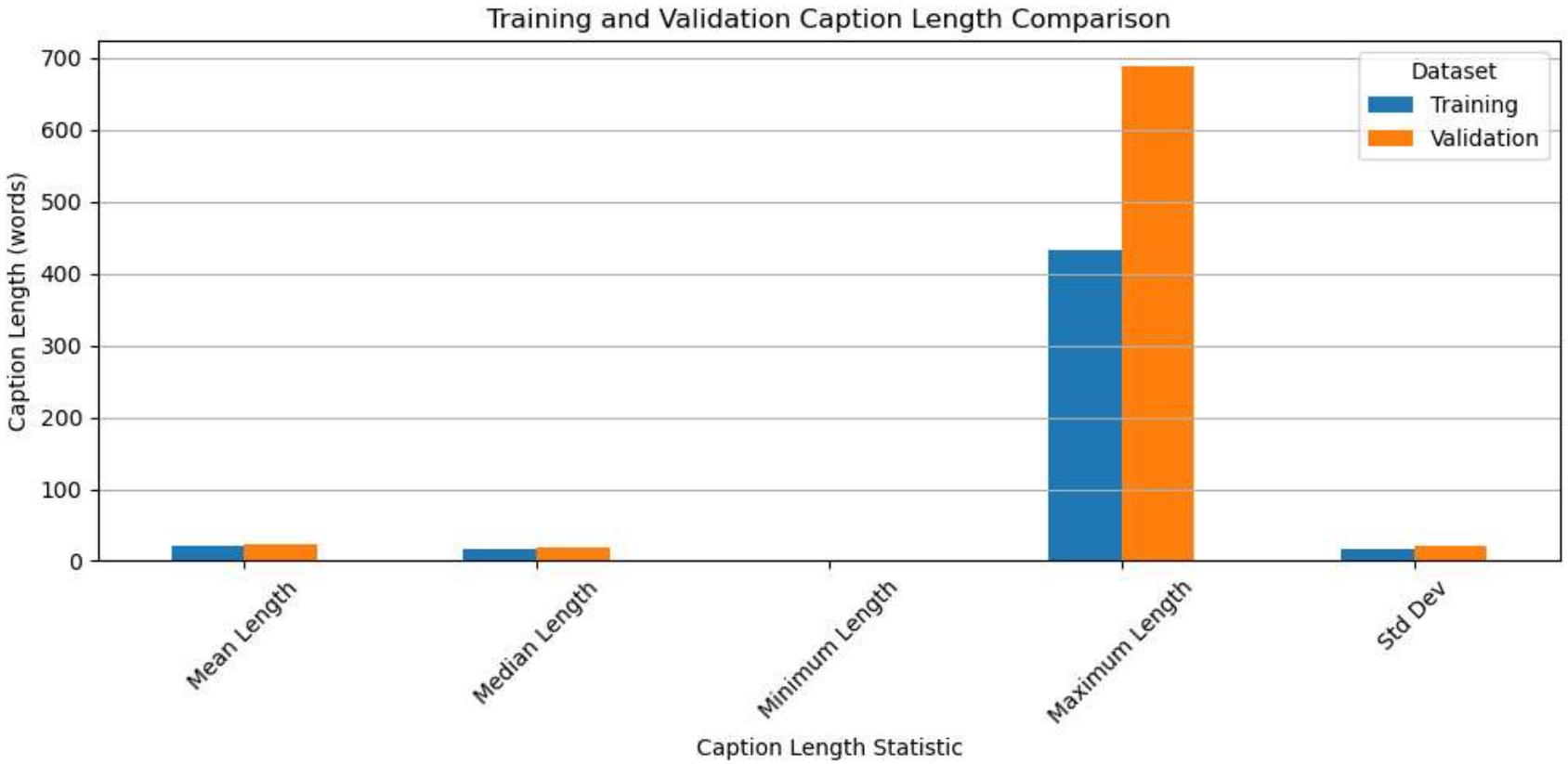
        # display plot
        plt.show()

        return (
            training_caption_lengths,
            validation_caption_lengths,
            caption_length_summary
        )

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [60]:

```
# calculate, summarise and compare training and validation caption lengths
training_caption_lengths, validation_caption_lengths, caption_length_summary =
plot_caption_length_comparison(training_captions, valid_captions)
```



In [61]:
```
# display caption length summary
display(caption_length_summary)
```

	Dataset	Captions	Mean Length	Median Length	Minimum Length	Maximum Length	Std Dev
0	Training	59958	21.378615	17.0	1	432	16.277227
1	Validation	9904	22.949616	18.0	0	689	21.833881

In [62]:
```
def plot_caption_length_loss(decoder, data_loader, device, bin_size=10):
    """
    Calculate and plot mean validation token loss

    Args:
        decoder: trained decoder model
```

```
        data_loader: validation DataLoader
        device: device used for model calculations
        bin_size (int): number of words in each caption-length range

    Returns:
        tuple:
            validation_caption_losses: individual caption-level loss results
            caption_length_loss_summary: mean loss grouped by caption-length range
    """

    try:

        # calculate individual caption losses and grouped summary
        validation_caption_losses, caption_length_loss_summary = (
            decoder.calculate_loss_by_caption_length(
                data_loader,
                device,
                bin_size=bin_size
            )
        )

        # create figure
        plt.figure(figsize=(10, 5))

        # plot mean token loss by caption-length range
        plt.plot(
            caption_length_loss_summary["Length Range"].astype(str),
            caption_length_loss_summary["Mean_Token_Loss"],
            marker="o"
        )

        # configure plot
        plt.xlabel("Caption Length (words)")
        plt.ylabel("Mean Token Loss")
        plt.title("Validation Loss by Caption Length")
        plt.xticks(rotation=45)
        plt.grid(True)

        # prevent labels from overlapping
        plt.tight_layout()

        # display plot
```

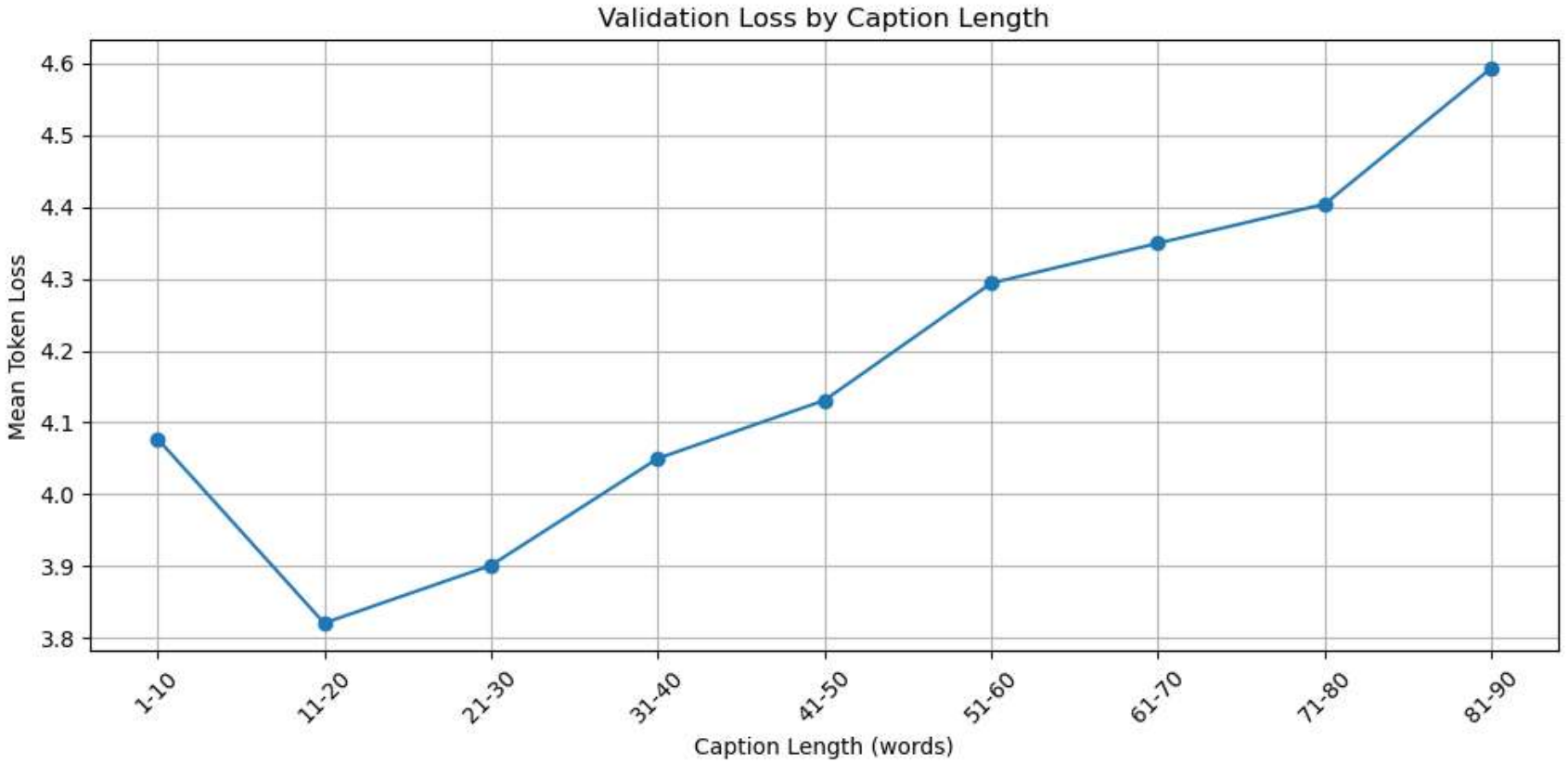
```
        plt.show()

        return (validation_caption_losses, caption_length_loss_summary)

    except Exception as e:
            raise RuntimeError(f"Exception : {str(e)}")
```

In [63]:
```
# calculate individual validation-caption loss statistics
validation_caption_losses, caption_length_loss_summary = (plot_caption_length_loss(decoder, valid_loader, device,
bin_size=10))
```

```
Loss By Caption Length:   0%|          | 0/310 [00:00<?, ?it/s]
```



In [64]:
```
# display vaidation caption losses
display(validation_caption_losses)
```

	Caption Length	Mean Token Loss	Total Caption Loss	Length Range
0	17	2.410179	43.383228	11-20
1	45	4.041842	185.924759	41-50
2	81	5.372726	440.563568	81-90
3	6	3.358365	23.508556	1-10
4	4	4.114731	20.573654	1-10
...	...	...	...	...
9899	22	3.720191	85.564400	21-30
9900	23	2.349306	56.383339	21-30
9901	44	3.604127	162.185699	41-50
9902	6	6.689893	46.829254	1-10
9903	80	5.031426	407.545532	71-80

9904 rows × 4 columns

In [65]:
```
# display caption lenght loss summary
display(caption_length_loss_summary)
```

	Length Range	Captions	Mean_Caption_Length	Mean_Token_Loss	Mean_Total_Caption_Loss
0	1-10	2115	7.134279	4.077560	32.799789
1	11-20	3570	15.245938	3.820828	62.019203
2	21-30	2077	24.937891	3.901131	101.349206
3	31-40	1022	35.006849	4.049854	145.917303
4	41-50	490	45.087755	4.130866	190.454668
5	51-60	259	55.355212	4.293925	242.005546
6	61-70	139	65.366906	4.349491	288.823932
7	71-80	80	75.487500	4.403964	337.047741
8	81-90	152	81.000000	4.593448	376.662762

References

Bahdanau, D., Cho, K. and Bengio, Y., Neural Machine Translation by Jointly Learning to Align and Translate (2015) Available At : https://arxiv.org/abs/1409.0473

Cruz Diaz, N.P. and Mana Lopez, M, 'An analysis of biomedical tokenization: problems and strategies', Proceedings of the Sixth International Workshop on Health Text Mining and Information Analysis', pages 40-49 (2015). Available At: https://aclanthology.org/W15-2605/

He, B., Guan, Y., Cheng, J., Cen, K. and Hua, W. CRFs Based De-identification Of Medical Records. Science Direct, 2015. Available at: https://doi.org/10.1016/j.jbi.2015.08.012

Moon, S.R., Pakhomov, S., Ryan, J. and Melton, G.B., 'Automated non-alphanumeric symbol resolution in clinical texts', AMIA Annual Symposium Proceedings (2011) pages 979–986. Available At : https://pmc.ncbi.nlm.nih.gov/articles/PMC3243158/

OCOM5100M Programming For Data Science

OCOM5101M Data Science

OCOM5200M Machine Learning

OCOM5203M Deep Learning

OCOM5203M Deep Learning Module, Summative Assessment 2

OCOM5204M Data Mining And Analytics

Project Jupyter, Jupyter Notebook documentation. Available At: https://jupyter-notebook.readthedocs.io/en/stable/

Python Software Foundation, Python 3.13 documentation (2026). Available At: https://docs.python.org/3.13/

Python Software Foundation, re — Regular expression operations (2026). Available At: https://docs.python.org/3/library/re.html

Robertson, S. (2024) NLP From Scratch: Translation with a Sequence to Sequence Network and Attention. PyTorch Tutorials. Available At: https://docs.pytorch.org/tutorials/intermediate/seq2seq_translation_tutorial.html

ROCOv2: Radiology Objects in Context Version 2, An Updated Multimodal Image Dataset. Available At : https://zenodo.org/records/10821435

sgrvinod, A PyTorch Tutorial to Image Captioning: Show, Attend, and Tell, GitHub (2026). Available At : https://github.com/sgrvinod/a-PyTorch-Tutorial-to-Image-Captioning

Xu, K.et al, Show, Attend and Tell: Neural Image Caption Generation with Visual Attention (2015). Available At :https://proceedings.mlr.press/v37/xuc15.html